Table of Contents:

Log In
View Main Menu Information
View and Manage Holiday
View Audit Log
View All Manufacturers
View Category
View Actual vs Predicted GPS
View Air Conditioners on Groundhog Day
View Store Revenue by Year by State
View District Volume by Category
View Revenue by Population

Login

Abstract Code

- User enters *employee ID* ($EmployeeID), *password* ($Password), which consists of the last 4 digits of their Social Security number, followed by a dash, followed by the properly capitalized last name, input fields.
- If data validation is successful for both *employee ID* and *password* input fields, then:
    - When ***Enter*** button is clicked:

      ```
      SELECT last_ssn, last_name FROM `User` WHERE employeeID=
      '$EmployeeID';
      ```

    - If User record is found but `CONCAT(User. last_ssn,'-', UPPER(User.last_name)) != '$Password'`:
        - Go back to **Login** form, with error message.
    - Else:
        - Store login information as session variable '$UserID'.
        - Go to **Main Menu** form.
- Else email and password input fields are invalid, display **Login** form, with error message.

View Main Menu Information

Abstract Code

The Main menu will serve as an interface for other pages, and contains hyperlinks for other tables.

- Once the user has logged into the system, execute the following SQL commands with the input *employee ID* ($ EmployeeID):

  ```
  SELECT first_name, last_name FROM `User` WHERE employeeID =
  '$EmployeeID';
  ```

  Then display the welcome message for the logged users with the extracted first_name and last_name.

- Followed by the welcome message, calculate the count of stores, cities, districts, manufacturers, products, categories, and holiday stored in the system and display the extracted results as a table:

```
SELECT
COUNT(DISTINCT Store.store_number),
COUNT(DISTINCT City.city_name),
COUNT(DISTINCT District.district_number),
COUNT(DISTINCT  Manufacture.manufacture_name),
COUNT(DISTINCT Product. PID),
COUNT(DISTINCT Category.category_name),
COUNT(DISTINCT Holiday.holiday_name)
FROM Store, City, District, Manufacture, Product, Category, Holiday;
```

- Display links for all the pages within the system, and if the user clicks certain link, then jump to the corresponding page.

View and Manage Holiday

Abstract Code;

- User clicked on ***Holiday Management Form*** button from **Main Menu.**
- If the user logged in has an *employeeID* with access to all districts (full_access is bool 1):

```
SELECT full_access
FROM `User`
Where employeeID = '$EmployeeID';
```

  - Run the **View Holidays** task: display business_date, holiday_name, employeeID, date_added in chronological order.

```
SELECT business_date, holiday_name, employeeID, date_added
FROM Holiday
ORDER BY holiday_name ASC;
```

  - Below the table displayed, provide an ***Add Holidays*** button.

- On the right side of each holiday information row within the table, provide a ***Delete*** button.
- When ***Add Holidays*** button is clicked:
  - Jump to Add Holidays window.
    - User enters business_d*ate(s)* ($Business_date), holiday_*name(s)* ($Holiday_name)
    - If the business_*date(s)* or the holiday_*name(s)* match(es) any of the business_date(s) or holiday_name(s) within the database, give an error message.

```
SELECT business_date, holiday_name
FROM Holiday
Where business_date = '$Business_Date ';
```

    - Else:
      - New Holiday(s) added to the Holiday table.

```
for each $Business_date, $Holiday_name
INSERT INTO Holiday (business_date, holiday_name, employeeID,
date_added) VALUES ('$Business_date', '$Holiday_name',
'$EmployeeID', NOW())
end for;
```

- When ***Delete*** button is clicked:
  - That row of holiday information is deleted.

```
DELETE FROM Holiday
WHERE business_date = $Bussiness_Date AND holiday_name =
$Holiday_Name;
```

- Otherwise:
  - Run the **View Holidays** task: display business_date, holiday_name, employeeID, date_added in chronological order.

```
SELECT business_date, holiday_name, employeeID, date_added
FROM Holiday
ORDER BY holiday_name ASC;
```

- When ready, user selects the next action from choices in **Main Menu**.

View Audit Log

Abstract Code

- Check if the current User using the system from the HTTP Session/Cookie has audit log permissions.

```
SELECT can_view_audit_log FROM `User`
WHERE employeeID = '$EmployeeID';
```

- If the User. can_view_audit_log != True:
  - Hide the ***View Audit Log*** button on **Main Menu.**
- Else:
  - Show ***View Audit Log*** button on **Main Menu.**
  - User clicked on ***View Audit Log*** button:
  - Run the **View Audit Log** task: query for the most recent 100 audit log records.

```
SELECT A.report_name, A.time_stamp, U.last_name, U.first_name,
CASE
WHEN U.employeeID IN (
                        SELECT Dp.employeeID
                        FROM DistrictPermission AS Dp
                        GROUP BY Dp.employeeID
                        HAVING COUNT(*) = (
                                SELECT COUNT(*)
                                FROM District
                        )
                )
                THEN '1'
                ELSE '0'
END AS full_access
FROM `AuditLog` AS A , `User` AS U
WHERE A.employeeID_view_report = U.employeeID
ORDER BY time_stamp DESC
LIMIT 100;
```

  - Display the report_name, timestamp, and the id of the log creator in the format of last_name and first_name separated by a comma and space.

    - If the User has access to all districts, mark the row with a yellow background.
- When ready, user selects a next action from choices in **Main Menu**.
- When any user views a report, execute the following SQLto create a new log, where '$EmployeeID' is the user who views the report, '$Time_stamp ' is the current time, and '$Report_name' is the name of the report being viewed.

```
INSERT INTO AuditLog ( employeeID, time_stamp,report_name)
VALUES ( '$EmployeeID', '$Time_stamp' , '$Report_name' );
```

View All Manufacturers

Abstract Code

- Run the **View All Manufactures** task: query for top 100 manufactures with the highest average price.
- Group the Product using the product manufacture and order by descending average retail price; Display the manufacture name(manufacture_name), total number of products(count), average retail price(avg_price), minimum retail price(min_price), and maximum retail price(max_price).

```
SELECT manufacture_name, COUNT(*) AS count, ROUND(AVG(retail_price),2)  AS
avg_price, MIN(retail_price)  AS min_price, MAX(retail_price)  AS max_price
FROM Product
GROUP BY Product.manufacture_name
ORDER BY avg_price DESC
LIMIT 100;
```

- When user clicks on each ***Manufacture Row***:
    - Display the manufacture name($Manufacture), total number of products(count), average retail price(avg_price), minimum retail price(min_price), and maximum retail price(max_price).
    - Find all the products using $Manufacture; Display the products' ID, name, category (or categories concatenated together, with a comma and space delimiting each category), and price, ordered by price descending (high to low).

```
SELECT Product.PID, pname, retail_price, GROUP_CONCAT(category_name
SEPARATOR ',  ') AS categories
```

```
FROM Product JOIN ProductCategory ON Product.PID =
ProductCategory.PID
WHERE Product.manufacture_name= '$Manufacture'
ORDER BY retail_price DESC
```

- When ready, user selects a next action from choices in **Main Menu**.

View Category

Abstract Code

- User clicked on **Category Report** button from **Main Menu**:
  To retrieve the category name, total number of products within each category, total number of manufacturers offering products in each category, and the average retail price (excluding discount days) of all the products within each category, sorted by category name in ascending order, the following SQL command can be executed and the results can be displayed as a table to view:

```
(SELECT
   c.category_name AS CategoryName,
   (SELECT COUNT(*)
    FROM Product p
    JOIN ProductCategory pc ON p.PID = pc.PID
    WHERE pc.category_name = c.category_name) AS Product_Number,
   (SELECT COUNT(DISTINCT p.manufacture_name)
    FROM Product p
    JOIN ProductCategory pc ON p.PID = pc.PID
    WHERE pc.category_name = c.category_name) AS Manufacturer_Number,
   (SELECT ROUND(AVG(p.retail_price) ,2)
    FROM Product p
    JOIN ProductCategory pc ON p.PID = pc.PID
    WHERE pc.category_name = c.category_name) AS AverageRetailPrice
FROM Category c
ORDER BY c.category_name ASC;
```

- When ready, user selects a next action from choices in **Main Menu**.

View Actual vs Predicted GPS

Abstract Code

- User clicked on **GPS Report** button from **Main Menu**:
- Run the **View Actual vs Predicted GPS** task: Query the database for a list of all CATEGORY.Name = "GPS". For each PRODUCT.PID:
  - Set:
    - $TotalQuantitySold = SUM(SOLD.Quantity)
    - $TotalDiscountedQuantitySold = SUM(CASE WHEN PRODUCT.Discount_price IS NOT NULL THEN SOLD.Quantity ELSE 0 END)
    - $TotalRetailQuantitySold = SUM(CASE WHEN PRODUCT.Discount_price IS NULL THEN SOLD.Quantity ELSE 0 END)
    - $ACTUAL_REVENUE = $TotalRetailQuantitySold * PRODUCT.Retail_price + $TotalDiscountedQuantitySold * PRODUCT.Discount_price)
    - $EXPECTED_REVENUE = (SUM(SOLD.Quantity) * 0.75) * PRODUCT.Retail_price
    - $ACTUAL_EXPECTED_DIFFERENCE = $ACTUAL_REVENUE - $EXPECTED_REVENUE
- Display in **Category Report**:
  - Product.PID
  - Product.Name
  - Product.Retail_price
  - $TotalQuantitySold
  - $TotalDiscountedQuantitySold
  - $TotalRetailQuantitySold
  - $ACTUAL_REVENUE
  - $EXPECTED_REVENUE
  - If $ACTUAL_EXPECTED_DIFFERENCE > 200, display $ACTUAL_EXPECTED_DIFFERENCE and sorted in descending order

```
SELECT
    p.PID AS ProductID,
    p.pname AS ProductName,
    p.retail_price AS RetailPrice,
    SUM(s.quantity) AS TotalQuantitySold,
```

```
    SUM(CASE WHEN d.discount_price IS NOT NULL THEN s.quantity ELSE 0
END) AS TotalDiscountedQuantitySold,

    SUM(CASE WHEN d.discount_price IS NULL THEN s.quantity ELSE 0 END)
AS TotalRetailQuantitySold,

    SUM(

        CASE

            WHEN d.discount_price IS NOT NULL THEN (d.discount_price *
s.quantity) + (p.retail_price * s.quantity)

            ELSE p.retail_price * s.quantity

        END

    ) AS ActualRevenue,

    SUM(s.quantity * 0.75 * p.retail_price) AS ExpectedRevenue,

    SUM(

        CASE

            WHEN d.discount_price IS NOT NULL THEN (d.discount_price *
s.quantity) + (p.retail_price * s.quantity)

            ELSE p.retail_price * s.quantity

        END

    ) - SUM(s.quantity * 0.75 * p.retail_price) AS ActualExpectedDifference

FROM    Sold s

JOIN    Product p ON s.PID = p.PID

JOIN    ProductCategory pc ON pc.PID = p.PID

JOIN    Discount d ON d.PID = p.PID

WHERE    pc.category_name = '$GPS'

GROUP BY    p.PID, p.pname, p.retail_price

HAVING    ActualExpectedDifference   > 200

ORDER BY    ActualExpectedDifference DESC;
```

- When ready, user selects next action from choices in **Main Menu**.

View Air Conditioner on Groundhog Day

Abstract Code

- User clicked on **Air Conditioners on Groundhog Day Report** button from **Main Menu**.
- Run the **View Air Conditioners on Groundhog Day** task: Query the database for a list of all CATEGORY.Name = ”Air Conditioner”, sorted by SOLD.Date.Year in ascending order.
    - For each SOLD.Date.Year:
        - Retrieve the total quantity ($TOTAL_QUANTITY = SUM(SOLD.Quantity)) of the PRODUCT sold in the year categorized by CATEGORY.Name = “Air Conditioner”
        - Set $AVE_PER_DAY = $TOTAL_QUANTITY / 365.0
        - Get the number of air conditioner products sold on February 2 of that year ($GHD_QUANTITY = SUM(SOLD.Quantity)) of products (CATEGORY.Name = “Air Conditioner” during that year, and the day is February 2)
        - Display SOLD.Date.Year, $TOTAL_QUANTITY, $AVE_PER_DAY, $GHD_QUANTITY in **Category Report**.

```
SELECT
   YEAR(s.business_date) AS Year,
   SUM(s.quantity) AS TotalQuantity,
   (SUM(s.quantity) / 365.0) AS AveragePerDay,
   (SELECT SUM(s2.quantity)
    FROM Sold s2
    JOIN Product p2 ON s2.PID = p2.PID
    JOIN ProductCategory pc2 ON p2.PID = pc2.PID
    WHERE pc2.category_name = '$air conditioning'
    AND s2.business_date = CONCAT(YEAR(s.business_date), '-02-02')) AS
GroundhogDayQuantity
FROM Sold s
JOIN Product p ON s.PID = p.PID
JOIN  ProductCategory pc ON p.PID = pc.PID
WHERE    pc.category_name = '$air conditioning'
```

```
GROUP BY    YEAR(s.business_date)
ORDER BY    YEAR(s.business_date) ASC;
```

- When ready, user selects next action from choices in **Main Menu**.

View Store Revenue by Year by State

Abstract Code

- When the button or *hyperlink* of **Store Revenue by Year by State Report** on **Main Menu** is clicked, run the **View Store Revenue by Year by State** task.
- Populate Drop-Down State:
  - Retrieve all distinct states from CITY table:

```
SELECT DISTINCT state
FROM City;
```

  - Populate the drop-down box with the retrieved states, allowing the user to select one *state* ($State).
- Once the *state* is selected run:
  - For each Sold table, join the Product table and the Store table, so now there are: store_number, city_id, PID, business_date, quantity, retail_price available to use, and the data are all the data for sold products.
  - Left outer join the above table with Discount table on business.date = Discount. business_date so we have discount_price on the dicount date.
  - Sum the revenue of each store for each product, when there is a discount_price, use discount_price, if not, use retail_price
  - Limit the State to "$state"
  - Group the summed data by store first so the result becomes the total revenue for each store, but not revenues for different products in each store.
  - Group the summed data by year for the next step so the result becomes the revenue by store, grouped by year.
  - Display the sorted results to the user.

```
SELECT Store.store_number, City.city_name, EXTRACT(YEAR FROM
Sold.business_date) AS year,

ROUND (SUM(

CASE
```

```
WHEN Discount.business_date = Sold.business_date THEN Sold.quantity *
Discount.discount_price
                        ELSE Sold.quantity * Product.retail_price
END
  ), 2) AS total_revenue
FROM Store
JOIN Sold ON Store.store_number = Sold.store_number
JOIN Product ON Product.PID = Sold.PID
JOIN City ON City.state = STORE.state AND City.city_name = STORE.city_name
LEFT JOIN Discount ON Discount.business_date = Sold.business_date
WHERE City.state = '$State'
GROUP BY Store.city_name, Store.store_number, year
ORDER BY year ASC, total_revenue DESC;
```

- When ready, user selects next action from choices in **Main Menu**.

View District Volume by Category

Abstract Code

According to the description of subtasks, after the corporate user has clicked the **District Volume for Category Report,** the platform should prompt the user to specify the time range that he or she wants to conduct the analysis. Then the report generation command executes.

- User enters the start yyyy-mm, $ StartDate and end date, $EndDate, that he or she wants to generate the report. Then the DBMS finds the district that sells the greatest number of units for each category.
- The SQL command for the task is as follows:

```
WITH DistrictCategory AS(
```

```
        SELECT ProductCategory.category_name, District.district_number,
        SUM(Sold.quantity) AS total_quantity

        FROM (Sold JOIN Product ON Sold.PID = Product.PID

        JOIN Store ON Sold.store_number = Store.store_number

        JOIN District ON Store.district_number = District.district_number

        JOIN ProductCategory ON Product.PID = ProductCategory.PID)

        WHERE Sold.business_date BETWEEN '2024-01' AND '2024-12'

        GROUP BY District.district_number, ProductCategory.category_name

)

SELECT DistrictCategory.category_name, DistrictCategory.district_number,
Max(DistrictCategory.total_quantity) as max_unit

FROM DistrictCategory

GROUP BY DistrictCategory.category_name

ORDER BY DistrictCategory.category_name ASC;
```

- Display the category name, the district that sold the highest number of units in that category, and the number of units that were sold by stores in that district, sorted by category name ascending.

- When user clicks on a row where the category is $category and district is $district to see details, execute the following commands.

```
SELECT Store.district_number, Store.store_number, Store.state, Store.city_name

FROM (Sold JOIN Product ON Sold.PID = Product.PID JOIN Store ON Sold.
store_number = Store.store_number JOIN District ON Store.district_number =
District.district_number JOIN ProductCategory ON Product.PID =
ProductCategory.PID)

WHERE  Sold.business_date BETWEEN $StartDate AND $ EndDate AND
TRIM(District.district_number) = $district AND ProductCategory.category_name=
$category

ORDER BY Store.store_number ASC
```

- Display header, including $category, year/month($StartDate AND $ EndDate), $district from the parent report.
- Display the store IDs, addresses, states, and cities of all the stores, ordered by store ID ascendingly.

- When ready, user selects the next action from choices in **Main Menu**.

View Revenue by Population

Abstract Code

- User clicked on ***View Revenue by Population*** button from **Main Menu**:
- Run the **View Revenue by Population** task: query for all cities, their population and revenue.
  - Find all cities, and CITY.Polulation.
  - Categorize these cities based on the population size as:
    - Small: population <3,700,000
    - Medium: 3,700,000 <= population <6,700,000
    - Large: 6,700,000 <= population <9,000,000
    - Extra Large: population >=9,000,000
  - For each city:
    - Find all stores located in this city.
    - For each store:
      - Find all sold_info where SOLD_INFO.Store == STORE.Store_number.
      - Group sold_info by year of SOLD_INFO.Date.
        - For each sold_info group:
          - Add PRODUCT.Selling_price to get annual revenue of this store.
        - Add each store's annual revenue to get the city's annual revenue.
  - For each population group:
    - Display population group name (Small/ Medium/ Large/ Extra Large).
    - Sort annual revenue of each city by year from oldest to newest and by population from smallest to largest; Display the CITY.Name, year and revenue.

```
SELECT
```

```
CASE
WHEN City.population < 3700000 THEN 'Small'
                WHEN City.population >=3700000 AND City.population < 6700000
THEN 'Medium'
                WHEN City.population >= 6700000 AND City.population < 9000000
THEN 'Large'
                ELSE 'Extra Large'
END AS city_size,
EXTRACT(YEAR FROM Sold.basiness_date) AS year,
        ROUND (SUM(
CASE
WHEN Discount.discount_price IS NOT NULL THEN Sold.quantity *
Discount.discount_price
                        ELSE Sold.quantity * Product.retail_price
END
         ) , 2) AS total_revenue
FROM Sold
JOIN Store ON Store.store_number = Sold.store_number
JOIN Product ON Product.PID = Sold.PID
JOIN City ON City.state = Store.state
LEFT JOIN Discount ON Discount.business_date = Sold.business_date
GROUP BY city_size, year
ORDER BY (
CASE
WHEN city_size = 'Small' THEN 1
WHEN city_size = 'Medium' THEN 2
                WHEN city_size = 'Large' THEN 3
```

```
                    ELSE 4
END
  ), year ASC;
```

- When ready, user selects the next action from choices in **Main Menu**.

IDF & EER DRAW.IO LINK:

Make all the changes in the phase2 graph. Leave phase1 as an outdated reference.

Phase2:

https://drive.google.com/file/d/1hgTYxPZyTkUs1u3cKVrpnD2X105tID0a/view?usp=sharing

Phase1:

https://drive.google.com/file/d/1Rrqe96MPQJaZRJLlegxw-mkx6KEA6f18/view?usp=sharing